Teoría React: Arquitectura de Componentes y Listas Dinámicas

1. ¿Qué es la Componentización?

La **componentización** es la estrategia de dividir una interfaz de usuario compleja en piezas pequeñas, independientes y reutilizables llamadas **Componentes**.

En nuestro proyecto "Landing React", en lugar de tener un archivo HTML de 500 líneas, tenemos piezas separadas como Navbar, Banner y Footer.

Beneficios:

- **Reutilización:** Puedes usar el mismo componente Button en diez lugares distintos.
- **Mantenibilidad:** Si hay un error en el formulario, solo buscas en ContactForm.tsx.
- **Separación de preocupaciones:** Cada componente se encarga de su propia lógica y estilo.

2. Jerarquía: Componentes Padres e Hijos

En React, los componentes se organizan en un "árbol". Un componente que contiene a otro se llama **Padre**, y el contenido es el **Hijo**.

La comunicación mediante Props

La información en React fluye en un solo sentido: **de arriba hacia abajo**.

- El **Padre** posee la información (estado o datos).
- El **Hijo** recibe la información a través de **Props** (propiedades), que funcionan como los argumentos de una función.

Ejemplo:

Si LandingPage (Padre) quiere que el Banner (Hijo) muestre un título específico, se lo pasa así:

<Banner title="Bienvenidos a mi Web" />

3. Renderizado de Listas: El método .map()

Cuando tenemos muchos elementos similares (por ejemplo, una lista de "Características" o "Servicios" en nuestra Landing), no escribimos el código manualmente para cada uno. Usamos el método de JavaScript .map().

¿Por qué .map()?

Transforma un array de datos (objetos) en un array de elementos visuales (JSX).

Ejemplo práctico en el proyecto:

Imagina que tenemos un array de servicios:

```
TypeScript

const services = [
  { id: 1, name: "Diseño Web" },
  { id: 2, name: "SEO" },
  { id: 3, name: "Marketing" }
];
```

En nuestro componente, lo recorremos así:

```
TypeScript

{services.map((service) => (
  <ServiceCard key={service.id} name={service.name} />
))}
```

La importancia de la key 🔑

Cuando usas .map(), React te obliga a poner una propiedad key única al elemento raíz del retorno.

- **¿Para qué sirve?** Ayuda a React a identificar qué elementos han cambiado, se han añadido o se han eliminado. Sin la key, React tendría que volver a renderizar toda la lista cada vez, lo cual es ineficiente.

4. Comparativa: Código "Limpio" vs "Sucio"

Característica	LandingPageBad (Mal camino)	LandingPage (Buen camino)
Organización	Todo el HTML en un solo archivo.	Dividido en carpetas por componente.
Repetición	Copias y pegas el mismo código de tarjeta 5 veces.	Usas .map() sobre un componente hijo.
Estilos	Un CSS gigante con conflictos de nombres.	**CSS Modules** (estilos privados por componente).
Escalabilidad	Imposible de mantener si crece.	Fácil de ampliar añadiendo nuevos componentes.

💡 Resumen para el alumno

1. **Piensa en componentes:** Antes de programar, dibuja en un papel qué partes de la web se repiten.
2. **Define el Padre:** El componente que orquesta la sección.
3. **Crea los Hijos:** Componentes pequeños que reciben datos por props.
4. **Usa .map():** Si ves que estás copiando y pegando el mismo código más de dos veces, probablemente necesites un array y un mapeo.